SwiftParcel ETA service

DA3408 AIOps, Assignment 5

Ramana Gopal Kodali DA24B010

Machine and setup

Platform	Linux aarch64 virtual machine; 4 logical CPUs
RAM / swap	7.6 GiB RAM; 4 GiB swap
Python	3.10.13 in the original `aiops` Conda environment
Serving	MLflow 3.17.0; Ray 2.40.0; FastAPI 0.115.6 and Starlette 0.41.3 for Ray
Conditions	Client and server shared the VM; 4 seconds per load-test level

The numbers below come from the original submitted output files, not the later rerun. The supplied `eta_data.py` reported fresh-data MAE of 11.95 min (v1) and 6.10 min (v2); the load sample contains the first 200 fresh rows and seven features. Environment evidence is in `evidence/environment.txt`. A separate clean checkout completed the README checks with Python 3.10.22 on Linux aarch64; timing differences from that rerun are not used below.

Part A — REST service

A.1. `partA/eta_app.py` exposes `app` and module-level `model`; it loads the bundle once at import from `ETA_MODEL`, defaulting to `data/eta_v1.joblib`. The prediction endpoint returns one-decimal ETAs in input order. Pydantic checks all seven feature ranges and batch size 1–500; distance above 150 km with more than 20 stops produces an infeasible HTTP 422. Unknown fields are rejected. Middleware echoes or generates `X-Request-ID`, also returned in JSON, including error responses. Validation errors contain `validation_failed`, field/message details and the request ID. The request counter and latency histogram exclude `/metrics` itself; `/healthz` reports status and version. Inference exceptions produce a generic 500 without internal details.

The live valid request returned HTTP 200 and 102.2 min; invalid distance and the infeasible combination returned 422. Request IDs matched their headers; health and metrics checks passed (`evidence/partA/a1_http_checks.txt`).

A.2. All **15 tests passed** in 0.95 s. They cover valid output, batch ordering, invalid distance/hour/rain, a missing feature, the cross-field boundary, batch sizes 0/1/500/501, echoed/generated IDs on success and errors, health, metric format/counting/exclusion, and a monkeypatched inference exception with no secret leaked. The deprecation warning did not fail the tests. Evidence: `evidence/partA/a2_pytest.txt`.

A.3.1. Load sweep: v1, one Uvicorn worker, batch 1, 4 s per level.

Clients	Requests	Req/s	p50 ms	p95 ms	p99 ms	Errors	$W = L/X$ ms
1	1652	413.0	1.9	4.6	5.5	0	2.42
2	2272	568.0	3.1	5.8	7.2	0	3.52
4	2170	542.5	6.9	12.7	17.1	0	7.37
8	2067	516.8	14.7	26.9	34.0	0	15.48
16	1961	490.2	30.6	53.4	62.8	0	32.64

A.3.2. Saturation and SLO capacity. The observed knee is around **2 clients**: throughput peaks at 568.0 req/s, then stays around 490–543 req/s as latency grows. By Little's Law, $L = XW$; treating the closed-loop concurrency as approximate requests in flight, implied mean time $W = L/X$ rises from 3.52 ms at 2 clients to 32.64 ms at 16. This is an approximate mean, not a percentile: once throughput stops growing, extra concurrency mainly adds waiting. Short runs and a shared client/server machine limit precision. The highest measured capacity under p95 $\leq$ 80 ms is **568.0 req/s**, at 2 clients and p95 5.8 ms. Every tested level meets the SLO; its breaking point was not located. Evidence: `partA/loadtest_results.json`.

A.3.3. Cost per row at concurrency 1.

Batch	Requests/s	Rows/s	Amortized ms/row
1	633.0	633	1.580
100	448.2	44,820	0.0223

Cost per row is 1000/(requests/s × batch size), an amortized wall-clock cost at one client. Batch 100 delivers about 70.8 times as many rows per second by sharing HTTP, JSON, validation, DataFrame and model-call overhead. Each row still waits for its complete batch response. These are throughput-derived costs, not isolated model-inference times. Evidence: `evidence/partA/a3_batch_comparison.txt`.

A.3.4. Applied change: two Uvicorn workers. I proposed a second worker to distribute inference across processes, then restarted with `-workers 2` and measured the same loads. It may roughly double capacity if a second CPU can process concurrent requests, but scaling is limited by contention and the shared load generator.

Clients	1 worker req/s	2 workers req/s	Speed-up	p95 before	p95 after	p99 after
1	413.0	657.2	1.59×	4.6	2.8	3.2
2	568.0	1091.5	1.92×	5.8	3.3	4.3
4	542.5	1261.2	2.32×	12.7	5.7	7.6
8	516.8	1347.5	2.61×	26.9	10.9	15.4
16	490.2	1083.8	2.21×	53.4	26.2	34.5

All two-worker levels had zero errors and p95 below 80 ms. Peak measured SLO capacity increased from 568.0 to **1347.5 req/s** (2.37×); the peak moves from 2 to 8 clients. At 16 clients, throughput falls, so the gain is not proportional at every load. The load tool's `replicas=1` field does not count Uvicorn workers; these responses do not identify separate workers. Evidence: `evidence/partA/a3_two_workers.json`.

A.3.5. Why p95/p99 rather than the mean? The mean can hide the slow requests that a minority of users experience. The p95/p99 quantify that tail and show what fraction of requests meets a latency target.

Part B — MLflow registry, gate and shadow

B.1. Registration. `EtaPyfunc` loads the bundle in `load_context` and returns a DataFrame with `eta_minutes` (one decimal) and `model_version`. Each model was logged with a signature, a three-row input example, and exact installed `pip_requirements` for MLflow, cloudpickle, joblib, NumPy, pandas, scikit-learn and SciPy. Both aliases were loaded and checked for output columns, row count and version in a fresh SQLite registry.

Version	Registered model	Alias	Bundle
1	eta-model	prod	v1
2	eta-model	candidate	v2

B.2. Promotion gate. Pure `decide()` requires at least 3% relative overall MAE gain, candidate single-row p95 at most 10 ms, and rush/rain MAE no more than 5% above prod. `promotion_gate()` uses `make_data(2000, seed=99)`, rush hours 7–9 and 16–19, rain=1, and 200 timed single-row pyfunc calls per model. Six tests passed: exact thresholds, four individual just-outside failures and a stricter gain override.

Metric	prod v1	candidate v2
Overall MAE (min)	11.9553	6.10485
Rush-hour MAE (min)	19.67986	8.57951
Rain MAE (min)	11.62274	6.28531
p95 ms, default evaluation	1.32901	1.12326
p95 ms, strict evaluation	0.99389	0.82363

Defaults: all four criteria true, `promote=true`; overall gain is 48.94%. **`min_rel_gain=0.60:`** only `overall_mae` false; other criteria true and `promote=false`. MAEs are unchanged across evaluations; timings vary. Outputs: `partB/gate_output.txt`; registration/tests: `evidence/partB/`.

B.3. Shadow comparison. The same first 200 rows of `eta_fresh.csv` were sent as single-row HTTP requests to prod on port 5101 and candidate on 5102. Every response had a finite ETA and the expected model version.

Measure	Original result
Matched requests per server	200
Mean absolute ETA difference	9.484 min
Difference greater than 10 min	70/200 (35.00%)
Prod HTTP p95	6.866 ms
Candidate HTTP p95	7.277 ms

I would advance v2 to a controlled canary because its default gate passes and overall MAE improves by 48.94%, with gains on both slices. Candidate HTTP p95 is 0.411 ms higher and 35% of ETAs differ by more than 10 minutes, so latency and actual delivery errors still need monitoring. Moving an MLflow alias does not update a server that already resolved and loaded the old version into memory. A rolling replacement can warm and health-check each new replica, route to it and drain the old replica's in-flight requests. A blue-green release can warm a complete new pool, switch routing to it and keep the old pool alive until pending requests finish. Evidence: `partB/shadow_output.txt`.

Part C — Ray Serve sticky canary

C.1. Application. `build(weight=0.1, extra_work_ms=0.0)` returns a Serve application. `EtaModel` loads once per replica; prod has two replicas and candidate autoscales from one to three. `EtaRouter` receives `customer_id` and `instances`, returning model output plus `arm`. It hashes the UTF-8 ID with SHA-256, maps the first eight bytes to an unsigned big-endian integer and chooses v2 below $\lfloor \text{weight} \times 2^{64} \rfloor$. `X-Beta: 1` forces v2 for that request. Each model replica and router reserves 0.25 CPU: at maximum, $2(0.25) + 3(0.25) + 0.25 = 1.5$ CPUs, fitting the 2-CPU cluster. These are scheduling reservations. `extra_work_ms` adds simulated work using sleep.

At weight 0, the ordinary smoke request returned HTTP 200, v1 and 102.2 min; the beta request returned HTTP 200, v2 and 103.9 min. The original FastAPI serialization failure was resolved with FastAPI 0.115.6 / Starlette 0.41.3 via a local overlay; the current README installs these directly in a fresh environment. Evidence: `evidence/partC/c1_smoke.txt`.

C.2. Verification. `verify_ray.py` ran on a 2-CPU Ray cluster and asserted the response version/arm for each check. All five passed; output is in `partC/verify_output.txt`.

Check	Observed result
(a) Share at weight 0.2	213/1000 = 21.3%, within 16–24%
(b) Repeat 200 customers	200/200 kept their original arm
(c) Delete and identical redeploy	200/200 kept their original arm
(d) Raise weight 0.2 to 0.5	v2 grew from 213 to 509; zero v2-to-v1 moves
(e) Beta override, 50 v1 customers	50/50 reached v2 with the header

C.3.1. Why not Python's `hash()`? String hashes are randomized between interpreter processes; different router replicas or restarts can assign a different bucket to the same customer. SHA-256 gives a repeatable mapping across those processes.

C.3.2. Why stickiness? For the product, a customer refreshing the same delivery should get consistent model behavior. For the experiment, keeping each customer in one arm reduces contamination from exposure to both models. The customer share is controlled; the request share can differ if customers issue different numbers of requests.

C.3.3. Autoscaling parameters. A target of two ongoing requests per candidate replica aims to add capacity before queues grow; `max_ongoing_requests=4` bounds concurrency, not latency. A five-second upscale delay reacts to sustained pressure. The 30-second downscale delay reduces churn after brief dips. Minimum one keeps v2 warm and maximum three limits resource use to the 2-CPU budget; these choices were configured, not validated under scaling load.

Part D — Release gate

D.1.1. Smoke test. `partD/smoke_eta.py` accepts `-url`, `-expect-version` and `-max-p95-ms` (default 80). Against the 10% Ray canary it performs seven checks: valid batch HTTP 200; expected arm/version; one prediction per input; finite numeric ETAs; positive fixture ETAs at most 1440 min; empty-batch HTTP 422; and HTTP p95 over 30 timed batch requests within the limit. The finite-value and plausible-range assertions are the implemented model-sanity checks; distance/traffic perturbation tests are not implemented.

The original v1 smoke run used a customer assigned to v1 and the v2 run used `-force-beta`. Both passed all seven checks; evidence is in `evidence/partD/d1_smoke_v1.txt` and `d1_smoke_v2.txt`, with the corresponding screenshot.

Model	Timed requests	HTTP p95	Limit	Result
v1	30	17.16 ms	80 ms	PASS
v2	30	12.79 ms	80 ms	PASS

The clean-checkout rerun separately verified exit code 0 for both successful model checks and exit code 1 when beta forced v2 while `-expect-version v1` was supplied. That confirms the intentional mismatch is blocked; its new timings do not replace the original table.

D.1.2. Four abort criteria during a 10% v2 canary. If any criterion triggers, set the canary weight to zero. Exclude beta overrides from ordinary rollout measurements, and disable the override too if v2 must receive no traffic. These are proposed rules; they were not exercised through an operational monitoring system.

Metric	Abort threshold	Observation window
Candidate availability	More than 1% of v2 attempts return HTTP 5xx or time out	Rolling 5 min; at least 100 v2 attempts
Candidate latency	Successful v2 HTTP p95 exceeds 80 ms	Rolling 5 min; at least 100 successful requests
Prediction sanity	Any nonfinite/nonpositive ETA or prediction/input count mismatch in a successful v2 response	Rolling 5 min; one occurrence suffices
Prediction quality	v2 MAE exceeds v1 MAE by more than 10% on the same deliveries	Rolling 24 h; at least 100 paired predictions with actual ETAs

What I did not test

- A separate load-generator machine, sustained-load stability or repeated sweeps with confidence intervals; levels ran for only 4 s. The SLO-breaking concurrency was not located.
- Prometheus multiprocess aggregation with two Uvicorn workers; the reported worker experiment measures HTTP performance, not a combined metrics view.
- Candidate autoscaling above one replica under load. C.2 checks routing and application redeployment, not VM restart, crash recovery or autoscaling performance.
- Real zero-drop model/alias replacement, operational rollback, or the proposed monitoring windows and alerts; these are design explanations and policies.
- Real delivery outcomes or execution on other operating systems. B.2 times in-process pyfunc calls; B.3 measures HTTP latency, without a concurrent shadow-load sweep.
- Dedicated just-inside cases for each gate criterion: the six-test suite includes exact and just-outside boundaries. The smoke test does not check distance/traffic monotonicity; systematic mutation testing was not performed.

The original evidence covers the implemented API, gate, shadow, canary and smoke checks. A clean checkout also completed the README workflow; those separate rerun logs are not part of the submitted measurement evidence. Short local tests do not establish production capacity or rollout safety.

Sources and AI use

DA3408 supplied assignment and starter files `eta_data.py`/`load_test.py`; Lecture 10 REST API, MLflow Serving and Ray Serve exercise notebooks: `https://github.com/sudarsun/DA3408`. ChatGPT assisted with implementation, debugging, interpretation and report drafting; details and attribution are in `AI_DISCLOSURE.md`, following the course AI fair-use policy. Reproduction is documented in `README.md`; exclude regenerated model bundles and databases from the submission ZIP.